Text generation

With the OpenAI API, you can use a [large language model](https://developers.openai.com/api/docs/models) to generate text from a prompt, as you might using [ChatGPT](https://chatgpt.com). Models can generate almost any kind of text response—like code, mathematical equations, structured JSON data, or human-like prose.

Here's a simple example using the [Responses API](https://developers.openai.com/api/docs/api-reference/responses), our recommended API for all new projects.

An array of content generated by the model is in the `output` property of the response. In this simple example, we have just one output which looks like this:

```json
[
  {
    "id": "msg_67b73f697ba4819183a15cc17d011509",
    "type": "message",
    "role": "assistant",
    "content": [
      {
        "type": "output_text",
        "text": "Under the soft glow of the moon, Luna the unicorn danced through fields of twinkling stardust, leaving trails of dreams for every child asleep.",
        "annotations": []
      }
    ]
  }
]
```

The `output` array often has more than one item in it! It can contain tool calls, data about reasoning tokens generated by [reasoning models](https://developers.openai.com/api/docs/guides/reasoning), and other items. It is not safe to assume that the model's text output is present at `output[0].content[0].text`.

Some of our [official SDKs](https://developers.openai.com/api/docs/libraries) include an `output_text` property on model responses for convenience, which aggregates all text outputs from the model into a single string. This may be useful as a shortcut to access text output from the model.

In addition to plain text, you can also have the model return structured data in JSON format—this feature is called [**Structured Outputs**](https://developers.openai.com/api/docs/guides/structured-outputs).

Prompt engineering

Prompt engineering is the process of writing effective instructions for a model, such that it consistently generates content that meets your requirements.

Because the content generated from a model is non-deterministic, prompting to get your desired output is a mix of art and science. However, you can apply techniques and best practices to get good results consistently.

Some prompt engineering techniques work with every model, like using message roles. But different models might need to be prompted differently to produce the best results. Even different snapshots of models within the same family could produce different results. So as you build more complex applications, we strongly recommend:

- Pinning your production applications to specific [model snapshots](https://developers.openai.com/api/docs/models) (like `gpt-5-2025-08-07` for example) to ensure consistent behavior
- Building [evals](https://developers.openai.com/api/docs/guides/evals) that measure the behavior of your prompts so you can monitor prompt performance as you iterate, or when you change and upgrade model versions

Now, let's examine some tools and techniques available to you to construct prompts.

Choosing models and APIs

OpenAI has many different [models](https://developers.openai.com/api/docs/models) and several APIs to choose from. [Reasoning models](https://developers.openai.com/api/docs/guides/reasoning), like o3 and GPT-5, behave differently from chat models and respond better to different prompts. One important note is that reasoning models perform better and demonstrate higher intelligence when used with the Responses API.

If you're building any text generation app, we recommend using the Responses API over the older Chat Completions API. And if you're using a reasoning model, it's especially useful to [migrate to Responses](https://developers.openai.com/api/docs/guides/migrate-to-responses).

Message roles and instruction following

You can provide instructions to the model with [differing levels of authority](https://model-spec.openai.com/2025-02-12.html#chain_of_command) using the `instructions` API parameter along with **message roles**.

The `instructions` parameter gives the model high-level instructions on how it should behave while generating a response, including tone, goals, and examples of correct responses. Any instructions provided this way will take priority over a prompt in the `input` parameter.

Generate text with instructions

```javascript
import OpenAI from "openai";
const client = new OpenAI();

const response = await client.responses.create({
    model: "gpt-5",
    reasoning: { effort: "low" },
    instructions: "${semicolonsDevMsg}",
    input: "${semicolonsPrompt}",
});

console.log(response.output_text);
```

```python
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5",
    reasoning={"effort": "low"},
    instructions="${semicolonsDevMsg}",
    input="${semicolonsPrompt}",
)

print(response.output_text)
```

```bash
curl "https://api.openai.com/v1/responses" \\
    -H "Content-Type: application/json" \\
    -H "Authorization: Bearer $OPENAI_API_KEY" \\
    -d '{
        "model": "gpt-5",
        "reasoning": {"effort": "low"},
        "instructions": "${semicolonsDevMsg}",
        "input": "${semicolonsPrompt}"
    }'
```

The example above is roughly equivalent to using the following input messages in the `input` array:

Generate text with messages using different roles

```javascript
import OpenAI from "openai";
const client = new OpenAI();
```

```
const response = await client.responses.create({
    model: "gpt-5",
    reasoning: { effort: "low" },
    input: [
        {
            role: "developer",
            content: "${semicolonsDevMsg}"
        },
        {
            role: "user",
            content: "${semicolonsPrompt}",
        },
    ],
});

console.log(response.output_text);
```

```python
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5",
    reasoning={"effort": "low"},
    input=[
        {
            "role": "developer",
            "content": "${semicolonsDevMsg}"
        },
        {
            "role": "user",
            "content": "${semicolonsPrompt}"
        }
    ]
)

print(response.output_text)
```

```bash
curl "https://api.openai.com/v1/responses" \\
    -H "Content-Type: application/json" \\
    -H "Authorization: Bearer $OPENAI_API_KEY" \\
    -d '{
        "model": "gpt-5",
        "reasoning": {"effort": "low"},
        "input": [
            {
                "role": "developer",
                "content": "${semicolonsDevMsg}"
            },
            {
                "role": "user",
                "content": "${semicolonsPrompt}"
            }
        ]
    }'
```

Note that the `instructions` parameter only applies to the current response generation request. If you are [managing conversation state](https://developers.openai.com/api/docs/guides/conversation-state) with the `previous_response_id` parameter, the `instructions` used on previous turns will not be present in the context.

The [OpenAI model spec](https://model-spec.openai.com/2025-02-12.html#chain_of_command) describes how our models give different levels of priority to messages with different roles.

<table>
  <thead>
    <tr>
      <th>developer</th>
      <th>user</th>
      <th>assistant</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>
        `developer` messages are instructions provided by the application
        developer, prioritized ahead of user messages.
      </td>
      <td>
        `user` messages are instructions provided by an end user, prioritized
        behind developer messages.
      </td>
      <td>
        Messages generated by the model have the <code>assistant</code> role.
      </td>
    </tr>
  </tbody>
</table>

A multi-turn conversation may consist of several messages of these types, along with other content types provided by both you and the model. Learn more about [managing conversation state here](https://developers.openai.com/api/docs/guides/conversation-state).

You could think about `developer` and `user` messages like a function and its arguments in a programming language.

- `developer` messages provide the system's rules and business logic, like a function definition.
- `user` messages provide inputs and configuration to which the `developer` message instructions are applied, like arguments to a function.

Reusable prompts

In the OpenAI dashboard, you can develop reusable [prompts](https://platform.openai.com/chat/edit) that you can use in API requests, rather than specifying the content of prompts in code. This way, you can more easily build and evaluate your prompts, and deploy improved versions of your prompts without changing your integration code.

Here's how it works:

1. **Create a reusable prompt** in the [dashboard](https://platform.openai.com/chat/edit) with placeholders like `{{customer_name}}`.
2. **Use the prompt** in your API request with the `prompt` parameter. The prompt parameter object has three properties you can configure:
   - `id` — Unique identifier of your prompt, found in the dashboard
   - `version` — A specific version of your prompt (defaults to the "current" version as specified in the dashboard)
   - `variables` — A map of values to substitute in for variables in your prompt. The substitution values can either be strings, or other Response input message types like `input_image` or `input_file`. [See the full API reference](https://developers.openai.com/api/docs/api-reference/responses/create).

<div data-content-switcher-pane data-value="simple">
    String variables
    Generate text with a prompt template

```javascript
import OpenAI from "openai";
const client = new OpenAI();

const response = await client.responses.create({
    model: "gpt-5",
    prompt: {
```

```
        id: "pmpt_abc123",
        version: "2",
        variables: {
            customer_name: "Jane Doe",
            product: "40oz juice box"
        }
    }
});

console.log(response.output_text);
```

```python
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5",
    prompt={
        "id": "pmpt_abc123",
        "version": "2",
        "variables": {
            "customer_name": "Jane Doe",
            "product": "40oz juice box"
        }
    }
)

print(response.output_text)
```

```bash
curl https://api.openai.com/v1/responses \\
  -H "Authorization: Bearer $OPENAI_API_KEY" \\
  -H "Content-Type: application/json" \\
  -d '{
    "model": "gpt-5",
    "prompt": {
      "id": "pmpt_abc123",
      "version": "2",
      "variables": {
        "customer_name": "Jane Doe",
        "product": "40oz juice box"
      }
    }
  }'
```

  <div data-content-switcher-pane data-value="filevar" hidden>
    Variables with file input
    Prompt template with file input variable

```javascript
import fs from "fs";
import OpenAI from "openai";
const client = new OpenAI();

// Upload a PDF we will reference in the prompt variables
const file = await client.files.create({
    file: fs.createReadStream("draconomicon.pdf"),
    purpose: "user_data",
});

const response = await client.responses.create({
    model: "gpt-5",
    prompt: {
        id: "pmpt_abc123",
        variables: {
            topic: "Dragons",
```

```
            reference_pdf: {
                type: "input_file",
                file_id: file.id,
            },
        },
    },
});

console.log(response.output_text);
```

```python
import openai, pathlib

client = openai.OpenAI()

# Upload a PDF we will reference in the variables
file = client.files.create(
    file=open("draconomicon.pdf", "rb"),
    purpose="user_data",
)

response = client.responses.create(
    model="gpt-5",
    prompt={
        "id": "pmpt_abc123",
        "variables": {
            "topic": "Dragons",
            "reference_pdf": {
                "type": "input_file",
                "file_id": file.id,
            },
        },
    },
)

print(response.output_text)
```

```bash
# Assume you have already uploaded the PDF and obtained FILE_ID
curl https://api.openai.com/v1/responses \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-5",
    "prompt": {
      "id": "pmpt_abc123",
      "variables": {
        "topic": "Dragons",
        "reference_pdf": {
          "type": "input_file",
          "file_id": "file-abc123"
        }
      }
    }
  }'
```

Next steps

Now that you known the basics of text inputs and outputs, you might want to check out one of these resources next.

[

    Use the Playground to develop and iterate on prompts.

](https://platform.openai.com/chat/edit)

[

    Ensure JSON data emitted from a model conforms to a JSON schema.

](https://developers.openai.com/api/docs/guides/structured-outputs)

[

    Check out all the options for text generation in the API reference.

](https://developers.openai.com/api/docs/api-reference/responses)